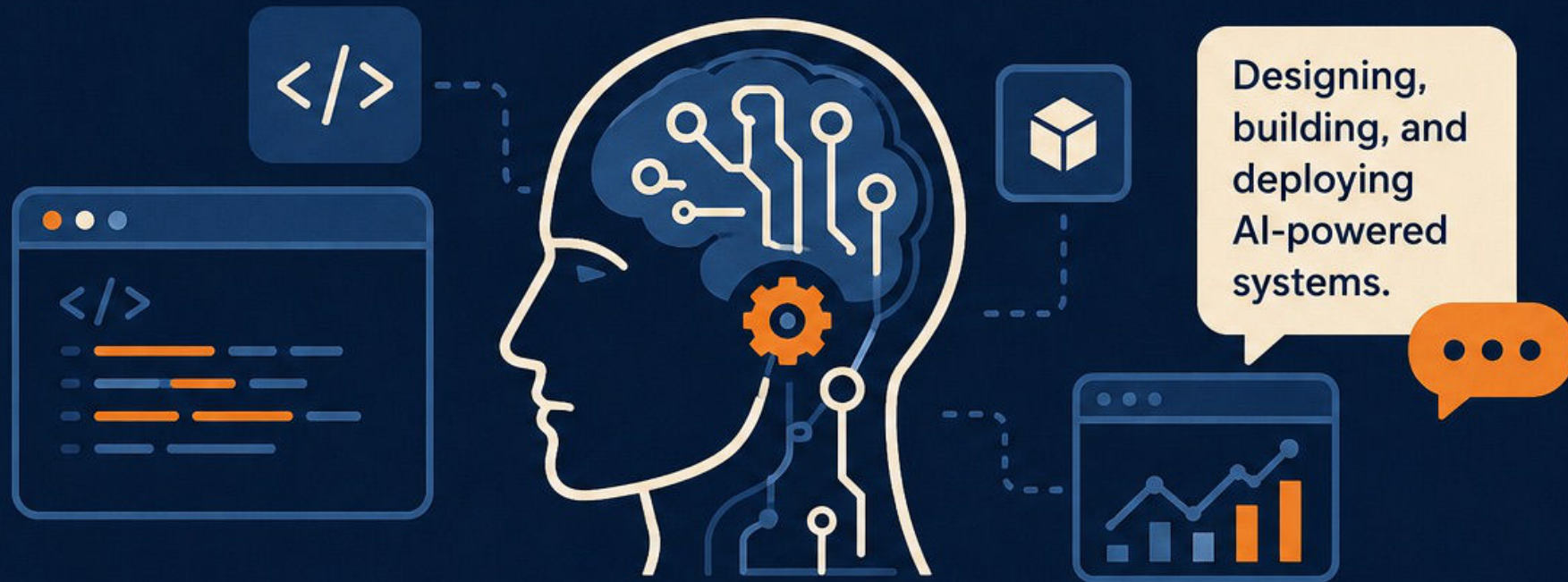


class starts
@ 9:05 pm

AI ENGINEERING



SYSTEM DESIGN



ML & AI
FUNDAMENTALS



DATA
ENGINEERING



MODEL
DEPLOYMENT



MLOPS



ETHICS &
RESPONSIBILITY

Lecture: DSPy - Prompt optimisation

~~~~~

# Agenda  
~~~~~

DSPy
~~~~~

→ Declarative Self-Improving python

# THE SCENARIO: YOUR FIRST MISSION



-  You joined a **fintech startup** on Monday.
-  Your task: build a news-sentiment agent that decides whether a headline is **bullish**, **bearish**, or **neutral**.
-  You wrote a prompt. It got **60% accuracy**.
-  Your CTO said: **"ship 80% by Friday or we cut the feature."**
-  You have **20 labeled examples** and no time.



**DEADLINE  
FRIDAY**

**TARGET  
80% ACCURACY**

## WHAT DO YOU DO?

### 1 Declare the Task (Signature)



- Define the task formally with DSPy **Signature**.
- Clear inputs, outputs and constraints.

```
class NewsSentiment(dspy.Signature):  
    """Classify news sentiment."""  
    headline: str = dspy.InputField()  
    sentiment: str = dspy.OutputField(  
        desc="bullish, bearish, or neutral"  
    )
```

### 2 Pick a Strategy (Module)



- Start with Chain-of-Thought to improve reasoning.
- Better accuracy without tuning prompts manually.

```
module = dspy.ChainOfThought(  
    NewsSentiment)
```

### 3 Evaluate (Baseline)



- Use your 20 labeled examples as validation set.
- Measure accuracy.

Baseline Accuracy:  
**~60%**

### 4 Optimize Fast (Teleprompter)



- Use BootstrapFewShot or MIPROv2.
- Let the optimizer find the best instructions + examples.

```
optimizer = dspy.MIPROv2(  
    metric=accuracy,  
    auto=True  
)  
optimized_module = optimizer.compile(  
    module, trainset=20_examples  
)
```

### 5 Ship & Iterate



- Get to 80%+ accuracy.
- Ship.
- Keep learning from new data.

Target:  
**80%+  
Accuracy**



### THE DSPY WAY

You don't guess the perfect prompt.  
You declare the task, choose a strategy, measure, and let the optimizer do the heavy lifting.



### RESULT:

From 60% to 80%+  
in a few hours, not days.



# Why does this happen?

Manual prompt engineering has **four structural problems**:

| Problem                                                                                                    | What it feels like                                                                                                                         |
|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Non-compositional</b> |  Fixing one failure mode <b>breaks two others</b>       |
|  <b>Non-portable</b>      |  Rewrites needed when you <b>swap models</b>            |
|  <b>Non-measurable</b>    |  You “ <b>feel</b> ” it’s better; you <b>don’t know</b> |
|  <b>Non-reproducible</b>  |  Your teammate <b>can’t reproduce</b> your prompt magic |

## ★ THE CORE INSIGHT



Prompts are **code**.

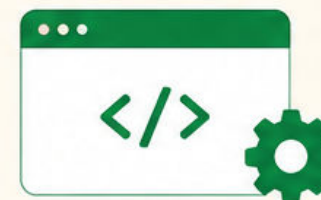


Writing them by hand  
is like writing  
**ASSEMBLY**.

```
mov r1, [mem]
cmp r1, r2
jne label
```



What we need is a  
**COMPILER**.



That compiler is **DSPy**.

# </> Analogy: Assembly vs High-Level Language



Think about how humans wrote software in the 1960s:



## 1 Assembly era (1960s)

- Programmers **hand-wrote CPU instructions**.
- Every model of CPU had **different assembly**.
- Portability was a **nightmare**.



Example  
(Assembly)

```
MOV R1, [MEM]
ADD R1, R2
CMP R1, #0
JNE LABEL
...
```



Prompt engineering today is in the **assembly era**.  
You are hand-writing model-specific strings and hoping they generalize.

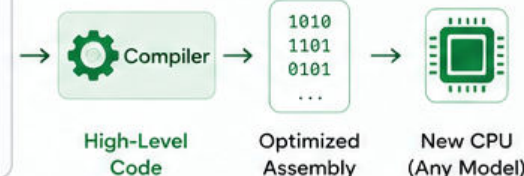


## 2 Compiler era (Now)

- You write in a **high-level language** (C, Python).
- The compiler figures out the assembly for you, **optimizes** it, and **re-optimizes** it when you target a new CPU.
- **Portability** and **optimization** come built-in.

Example  
(High-Level)

```
int sum(int *a, int n) {
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    return s;
}
```



You write once. The compiler handles the details, optimizes, and adapts to the target.



## DSPy is the compiler era for LLMs.

1

### Declare (Signatures)



You write high-level declarations of what you want.

```
class Classify(dspy.Signature):
    headline: str = dspy.InputField()
    sentiment: str = dspy.OutputField()
```

2

### Choose Strategy (Module)



You pick a strategy for how to get there.

```
module = dspy.ChainOfThought(Classify)
# or Predict, ReAct, ProgramOfThought, etc.
```

3

### Compile (Teleprompter)



You let a teleprompter (optimizer) compile the actual prompt against your data.

```
optimizer = dspy.MIPROv2(metric=accuracy)
optimized_module = optimizer.compile(
    module, trainset=examples
)
```

4

### Recompile on Change



When the model changes, you just recompile.

```
# Swap model
lm = dspy.LM('new-model')
# Recompile
optimized_module = optimizer.compile(
    module, trainset=examples
)
```



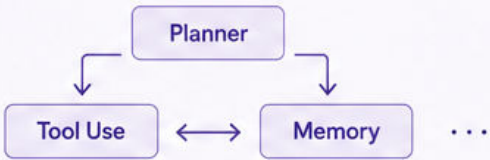
From hand-writing model-specific strings to **declare intent** and letting a compiler **produce**, **optimize**, and **adapt**.



# Where DSPy fits in the agentic AI stack



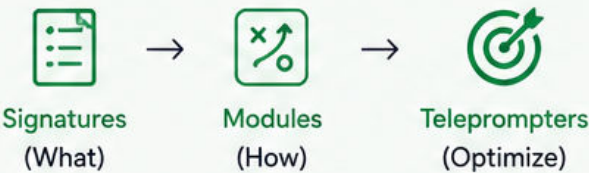
**Agentic Framework**  
(LangGraph, CrewAI, AutoGen)



**Orchestration**  
Plans, routes, and coordinates steps, tools, and memory.



**Prompt Optimization**  
(DSPy)



**What we learn today**  
Turns your declarations into optimized prompts using your data and metrics.



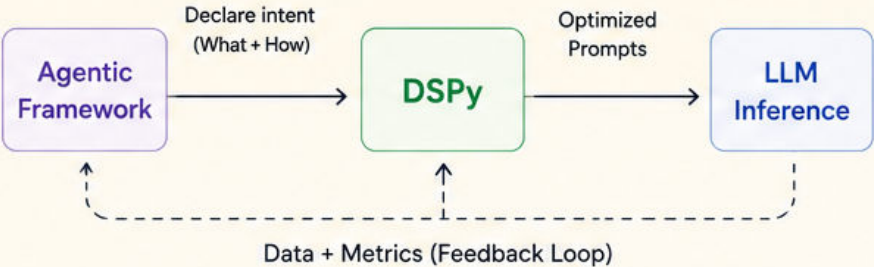
**LLM Inference**  
(Groq, OpenAI, Anthropic, ...)



**Raw model calls**  
Executes the prompt on the model and returns the completion.



**DSPy sits between your agent framework and the raw model.**  
It converts your declarations into model-specific prompts that are algorithmically tuned on your data.



**Why this placement matters**



**Better Performance**  
Prompts are tuned on your data, not guessed.



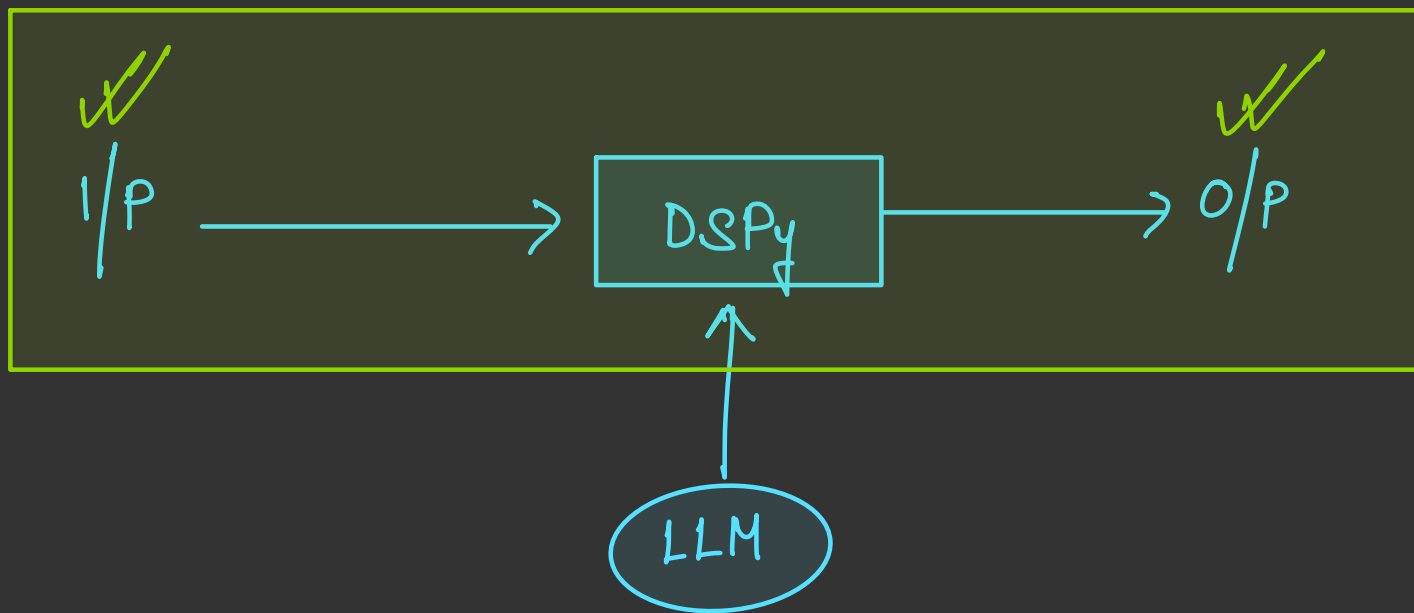
**Model Portability**  
Recompile when models change—no rewrites.



**Measurable & Reproducible**  
Optimization is metric-driven and repeatable.



**Scales with Complexity**  
Works for simple tasks and large agentic workflows.



signature  
Task + I/P +  
O/P

DSPy stands for **Declarative Self-improving Python**. It came out of Stanford NLP (Omar Khattab et al.) and is now the de-facto framework for **programming language models** instead of prompting them.

## THE THREE PILLARS OF DSPy



### 1 Signatures

A typed declaration of an LLM task.  
Not a prompt, a **specification**.

```
class Classify(dspy.Signature):  
    "Classify sentiment of financial news."  
    headline: str = dspy.InputField()  
    sentiment: str = dspy.OutputField(  
        desc="one of: positive, negative, neutral"  
    )
```



Think of it as a function  
**type hint** for an LLM call.



### 2 Modules ✓

A **strategy** for executing a signature.  
Same signature, different modules,  
different behaviors.

- `dspy.Predict(Classify)`: single-shot classification
- `dspy.ChainOfThought(Classify)`: forces reasoning before answer
- `dspy.ReAct(Classify, tools=[...])`: tool-using agent
- `dspy.ProgramOfThought(Classify)`: generates Python code as reasoning



Think of it as choosing an  
**implementation strategy** for  
the declared function.



### 3 Teleprompters (Optimizers)

The **compiler**. Given a module, a metric,  
and training data, it produces an  
**optimized** version of the module.

- **BootstrapFewShot**: learns which examples to include in the prompt
- **BootstrapFewShotWithRandomSearch**: searches over example combinations
- **MIPROv2**: jointly optimizes instructions AND examples using Bayesian-style search
- **BootstrapFinetune**: turns your prompt program into a fine-tuning job



Think of it as the **compiler** that turns  
your declaration + strategy into an  
actual working prompt.

## THE MENTAL MODEL



1. Signature  
(What)



2. Module  
(How)



3. Teleprompter  
(Optimize)



Optimized  
LLM Program



DSPy lets you declaratively **define**,  
strategically **execute**, and automatically  
**improve** LLM programs.



# The mental model shift

| Old world (Prompting)                                                             |                                     | New world (DSPy) |                                                                                                                             |
|-----------------------------------------------------------------------------------|-------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------|
|  | Write a string                      | →                |  Declare a <b>Signature</b>              |
|  | Copy-paste examples into the string | →                |  Provide <b>training data</b> separately |
|  | Guess at wording                    | →                |  Pick a <b>Module</b> strategy           |
|  | Eyeball outputs                     | →                |  Define a <b>Metric</b>                  |
|  | Re-write on model swap              | →                |  <b>Recompile</b>                        |



## Analogy: DSPy is to prompts what PyTorch is to neural nets

### PyTorch

Before PyTorch, people wrote raw NumPy for backprop. It worked, but nobody could scale it or share it.

```
# Raw NumPy backprop
for i in range(L-1, -1, -1):
    dz = ...
    dw = ...
    db = ...
    W[i] -= lr * dw
    b[i] -= lr * db
```

#### PyTorch gave us:



**nn.Module**  
for declaration



**optim.SGD**  
for optimization



**Autograd**  
for automatic  
differentiation



### DSPy

DSPy gives us the same kind of abstraction for prompt engineering and LLM programs.

```
class Classify(dspy.Signature):
    """Classify news sentiment."""
    headline: str = dspy.InputField()
    sentiment: str = dspy.OutputField()
```

#### DSPy gives us:



**dspy.Signature**  
for declaration



**dspy.Teleprompter**  
for optimization



**Bootstrapped  
demonstrations**  
for automatic  
prompt search



Same abstraction shift, **different domain**.



Neural nets



Prompts



# THE CORE IDEA

A “prompt” is really a combination of:

-  **Instruction**  
(the docstring / system message)
-  **Demonstrations**  
(few-shot examples)
-  **Output format**  
(structure constraints)

DSPy treats this as a **discrete optimization problem**:

$$prompt^* = \arg \max_{prompt \in \mathcal{P}} \mathbb{E}_{(x,y) \sim \mathcal{D}_{val}} [\mathcal{M}(f_{prompt}(x), y)]$$

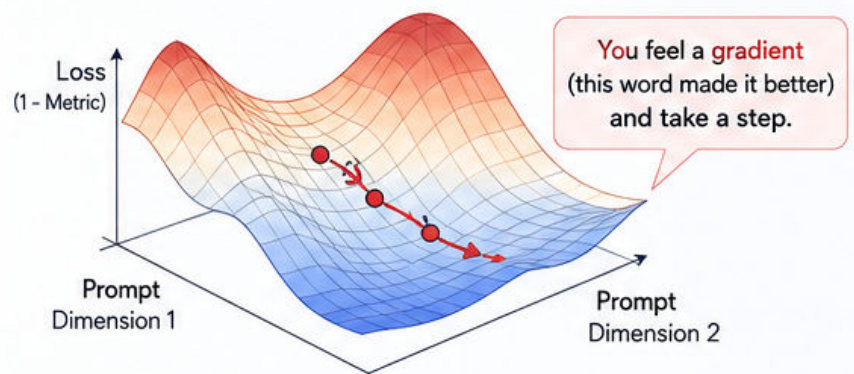
Where:

- $\mathcal{P}$  = space of possible prompts (instructions + demo combinations)
- $\mathcal{D}_{val}$  = validation data distribution
- $\mathcal{M}$  = metric function (our sentiment\_match)
- $f_{prompt}$  = LLM applied with a specific prompt

 The teleprompter is the **search algorithm** over  $\mathcal{P}$ .

## THE MATH INTUITION

Manual prompt engineering is **gradient descent by human intuition**.



But:

-  You **can't see** the loss landscape.
-  You can only sample **one point** at a time.
-  Your feedback signal (5 test examples) is **noisy**.

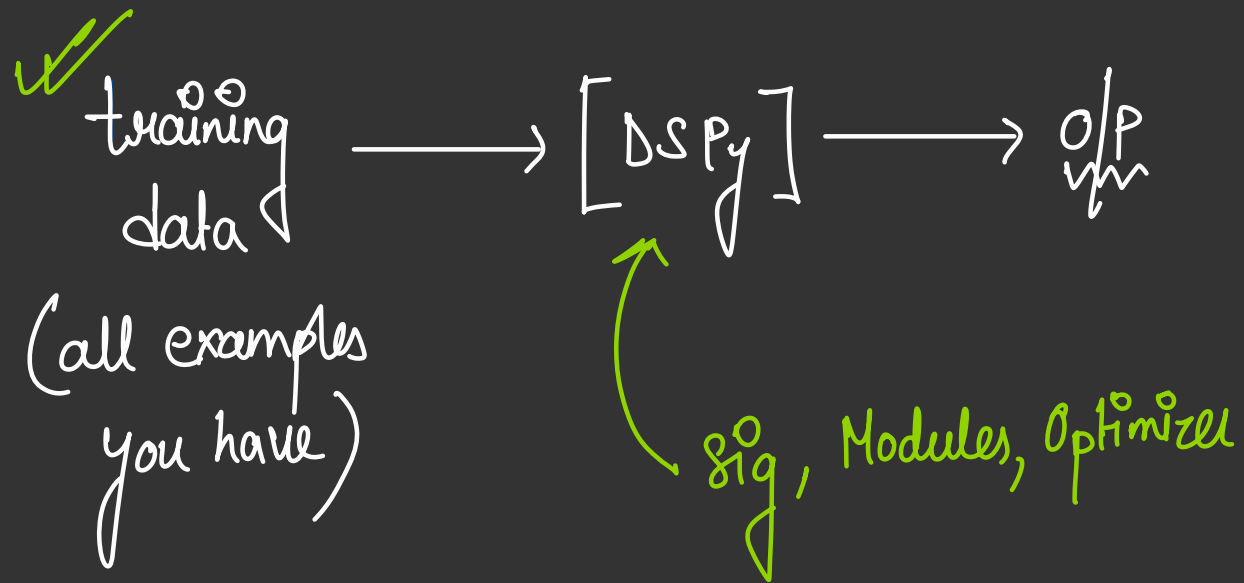


**Teleprompters do this systematically,**  
with a real metric on a real held-out set.

- ✓ Search the space  $\mathcal{P}$  efficiently
- ✓ Use a real metric  $\mathcal{M}$
- ✓ Evaluate on held-out  $\mathcal{D}_{val}$



DSPy converts prompt engineering from **guessing** to **optimization**.





prompt x

(DSPy)

examples (labelled data)

```
graph LR; ex[examples (labelled data)] --> DSPy((DSPy))
```

label data

Train a DSPy

```
graph LR; ld[label data] --> train[Train a DSPy]
```



get me a system prompt → Don't pass

```
graph LR; subgraph prompt [get me a system prompt]; end; prompt --> fail[Don't pass]
```

i/p → o/p  
Text Free text

